



MÓDULO 5: FLUJO DE DESARROLLO CON IA

PROMPT ENGINEERING PARA DEVELOPERS

BIG school



Prompt engineering para developers

BIG school

Índice

- Principios del prompt engineering aplicados a código
- Estructuras efectivas de prompts para depuración, generación y optimización
- Uso de cadenas de prompts (prompt chaining)
- Roles y personificación de IA para objetivos de desarrollo
- Taller práctico de prompts con Codex
- Conclusiones

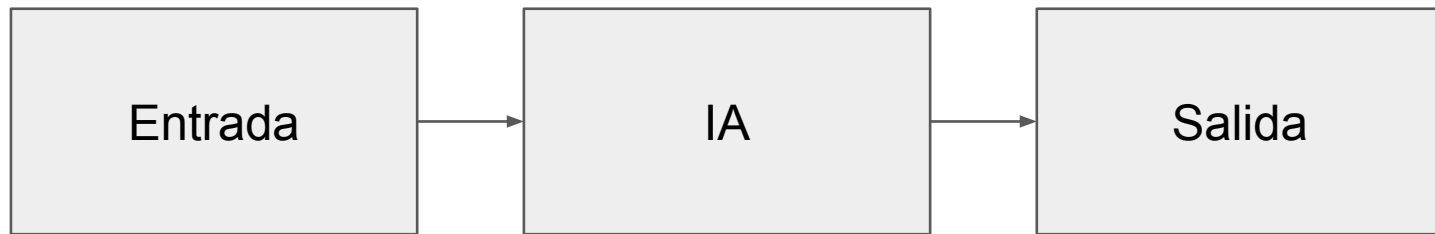
MÓDULO 5: FLUJO DE DESARROLLO CON IA

PRINCIPIOS DEL PROMPT ENGINEERING APLICADOS A CÓDIGO

PROMPT ENGINEERING PARA DEVELOPERS

BIG school

Prompt $\approx$ Código



Prompt $\approx$ Código

Prompt Vago:

"Hazme un login en JS"

Prompt Mejorado:

"Escribe una función loginUser(email, password) en Node.js usando bcrypt para hashear la contraseña y JWT para la sesión. Devuélveme el código comentado."

Principio 1: Sé específico

Prompt Vago:

"Hazme un test"

Prompt específico:

"Genera un test unitario con pytest para la función 'calculate_tax', probando casos borde como valores negativos y un input vacío."

- Precisión en el **objetivo de la tarea**.
- Detalles sobre *qué debe hacer* la IA.

Principio 2: Da contexto

Sin contexto:

"Hazme un endpoint de usuarios"

Con contexto:

"Crea un endpoint en Express.js llamado /users que devuelva usuarios desde MongoDB usando Mongoose. Devuélvelo como código listo para copiar en un archivo routes.js"

- El **entorno, tecnologías, restricciones** donde el código debe funcionar.
- Información que rodea a la tarea para que la IA genere algo aplicable.

Principio 3: Formato de salida

Sin formato:

"Explícame este código"

Con formato:

"Explícame este código en bullets de máximo 1 línea cada uno y genera un diagrama de flujo en Markdown."

- El **entorno, tecnologías, restricciones** donde el código debe funcionar.
- Información que rodea a la tarea para que la IA genere algo aplicable.

Principio 4: Iteración

Dev: "Refactoriza este código para hacerlo más legible."

IA: devuelve versión optimizada

Dev: "Ahora optimízalo para rendimiento en arrays grandes."

IA: devuelve nueva versión mejorada

Dev: "Genérame tests unitarios para validar esta función."

Ejemplo de prompt completo

Quiero que actúes como un desarrollador senior en Node.js.

Necesito una función `loginUser(email, password)` que:

- Use `bcrypt` para comparar la contraseña
- Genere un JWT válido de 1h de duración
- Devuelva errores claros si las credenciales fallan

Incluye:

1. Código comentado
2. Un test unitario en Jest cubriendo casos correctos e incorrectos
3. La salida estructurada en bloques de código separados



MÓDULO 5: FLUJO DE DESARROLLO CON IA

ESTRUCTURAS EFECTIVAS DE PROMPTS PARA DEPURACIÓN, GENERACIÓN Y OPTIMIZACIÓN

PROMPT ENGINEERING PARA DEVELOPERS

BIG school

¿Por qué usar estructuras?

-  Prompt desordenado = respuestas inconsistentes.
-  Prompt estructurado = resultados claros y repetibles.
- “Los prompts son como funciones: si defines bien la interfaz, el output es predecible.”

Prompt para depuración (Debugging)

Prompt vago:

"Encuentra el error en este código"

Prompt estructurado:

"Analiza este código paso a paso:

- 1. Señala el error principal*
- 2. Explica la causa en 2 frases*
- 3. Propón una solución con snippet corregido"*

Prompt para generación de tests

Prompt vago:

"Genera tests"

Prompt estructurado:

"Genera un set de tests unitarios para esta función en pytest. Incluye:

- Caso normal*
- Casos borde*
- Manejo de errores*

Devuelve los tests listos para copiar en test_calculate.py"

Prompt para optimización de código

Prompt vago:

"Optimiza este código"

Prompt estructurado:

"Optimiza esta función en JS para arrays grandes:

- 1. Identifica problemas de rendimiento*
- 2. Propón una versión optimizada*
- 3. Explica la mejora en 2 frases"*

Plantilla general reutilizable

Actúa como [rol: QA tester / senior dev / optimizador].

Quiero que realices [tarea: depuración / generar tests / optimizar].

Aquí tienes el código/contexto: [pegar fragmento].

Tu respuesta debe incluir:

- 1. [Explicación breve en bullets]*
- 2. [Código corregido o generado]*
- 3. [Validación o recomendaciones extra]*

-  Rol
-  Tarea
-  Contexto
-  Formato de salida

De un prompt vago a uno estructurado

Vamos a crear un test para esta función:

```
// Código base  
function calculateDiscount(price, percentage) {  
  return price - (price * percentage / 100);  
}
```

No te obsesiones con el prompt perfecto

“Las estructuras ayudan... pero los modelos mejoran cada día.”

“No se trata de memorizar formatos, sino de **entender cómo guiar a la IA.**”



MÓDULO 5: FLUJO DE DESARROLLO CON IA

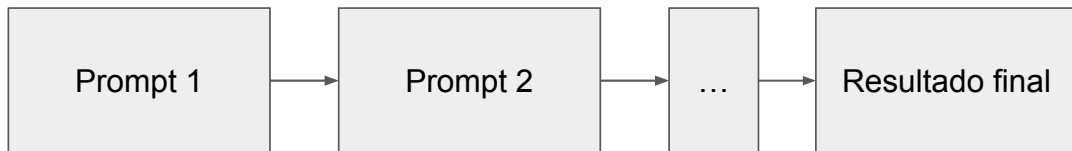
USO DE CADENAS DE PROMPTS (PROMPT CHAINING)

PROMPT ENGINEERING PARA DEVELOPERS

BIG school

¿Qué es el Prompt Chaining?

- Encadenar varios prompts para resolver una tarea compleja.
- Cada prompt → un paso del proceso.
- Ventajas: más control, menos errores, resultados iterativos.
- Similar a *divide y vencerás* aplicado a la IA.



Un prompt vs varios prompts encadenados

Prompt único:

"Hazme una app de notas completa con login, frontend y backend"

Prompt encadenado

- 1. Define requisitos de la app de notas*
- 2. Crea la arquitectura básica (frontend + backend)*
- 3. Genera el endpoint de login*
- 4. Añade un CRUD de notas*
- 5. Genera tests unitarios para el CRUD*
- 6. Documenta la API en Markdown*

Casos: En el desarrollo de una feature

-  Prompt 1 → Define requisitos de la feature.
-  Prompt 2 → Genera el endpoint en Express.js.
-  Prompt 3 → Genera tests unitarios en Jest.
-  Prompt 4 → Documenta la API en Markdown.

Casos: En migración de código legacy

-  Prompt 1 → *Explica el código existente paso a paso.*
-  Prompt 2 → *Refactoriza el código a una versión más moderna.*
-  Prompt 3 → *Optimiza rendimiento y elimina redundancias.*
-  Prompt 4 → *Genera tests unitarios para validar la nueva versión.*

Casos: para aprender paso a paso

-  Prompt 1 → *Explícame qué es useState con un ejemplo sencillo.*
-  Prompt 2 → *Muéstrame un ejemplo práctico en una app de contador.*
-  Prompt 3 → *Ahora compáralo con useReducer.*
-  Prompt 4 → *Hazme un quiz de 5 preguntas para practicar.*

Casos: Depuración compleja

-  Prompt 1 → *Analiza este stack trace y dime qué significa.*
-  Prompt 2 → *Propón una posible solución.*
-  Prompt 3 → *Genera un fix para el archivo index.js.*
-  Prompt 4 → *Crea un test unitario que valide que el error no vuelva a ocurrir.*

Resumen

- Divide tareas grandes en pasos pequeños.
- Valida resultados antes de avanzar.
- Encadena prompts = más control y menos errores.
- Piensa en flujos, no en instrucciones aisladas.

MÓDULO 5: FLUJO DE DESARROLLO CON IA

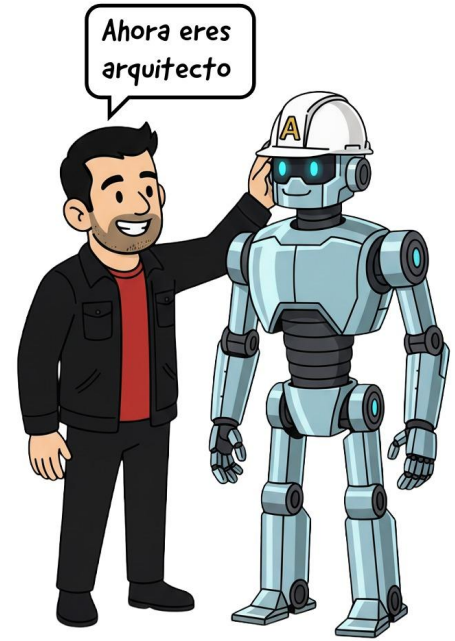
ROLES Y PERSONIFICACIÓN DE IA PARA OBJETIVOS DE DESARROLLO

PROMPT ENGINEERING PARA DEVELOPERS

BIG school

¿Qué significa dar un rol a la IA?

- Indicarle qué papel debe asumir (ej. “*Sé un revisor de código senior*”).
- Cambia el **tono**, el **nivel de detalle** y el **enfoque** de la respuesta.
- Permite alinear el output con lo que necesitas en cada momento.
- Es como “contratar” distintos perfiles, pero dentro de la IA.



Roles útiles para developers



Developer → Genera código siguiendo buenas prácticas.



Tester → Diseña casos de prueba y escenarios límite.



Arquitecto → Propone estructuras y patrones de diseño.



Profesor → Explica conceptos paso a paso con ejemplos.



Auditor → Encuentra problemas de seguridad o calidad.

Ejemplo práctico: Sin rol vs con rol

Prompt sin rol:

"Hazme un sistema de login en Node.js"

Prompt con rol

*"Actúa como un arquitecto de software senior:
Diseña un sistema de login en Node.js con JWT,
bcrypt y separación por capas siguiendo buenas
prácticas."*

¿Siguen siendo útiles los roles?

- Hoy en día los modelos entienden mucho mejor el contexto.
- Los roles importan menos que antes.
- Pero siguen siendo un recurso práctico para orientar la respuesta.
- Úsalos cuando quieras un tono, estilo o enfoque específico.



MÓDULO 5: FLUJO DE DESARROLLO CON IA

TALLER PRÁCTICO DE PROMPTS CON CODEX: TO-DO LIST APP

PROMPT ENGINEERING PARA DEVELOPERS

BIG school

To-do list app

Haremos una app de To-do list promptando a Codex.

MÓDULO 5: FLUJO DE DESARROLLO CON IA

CONCLUSIONES

PROMPT ENGINEERING PARA DEVELOPERS

BIG school

GENERAMOS
NEGOCIO

BIGSEO

Lo que aprendiste

-  Ser **claro y específico** en los prompts.
-  Usar **estructuras efectivas** para debugging, generación y optimización.
-  Aplicar **Prompt Chaining** para dividir tareas grandes en pasos pequeños.
-  Probar **roles/personificación** cuando quieras orientar el tono o el enfoque.
-  Iterar y ajustar: los prompts se **depuran igual que el código**.

Claves prácticas para tu día a día

-  Piensa en los prompts como **funciones** → input claro, output esperado.
-  **Divide tareas grandes** en pasos pequeños y manejables.
-  **Valida los resultados:** no aceptes el código sin revisarlo.
-  Usa la IA como **copiloto**, no como piloto automático.

***“La IA no sustituye al developer, lo potencia.
La diferencia está en cómo sabes hablar su lenguaje.”***

